

nitevow VPS Deployment Checklist

Target: VPS 1853920

Repository: Dandy-S00/nitevow

1. Confirm the VPS prerequisites

- VPS 1853920 is active and reachable over SSH.
- Ubuntu and Docker are installed and working.
- Docker Compose is available with `docker compose version`.
- The VPS has enough free disk space for the source tree, Docker layers, MySQL data, and logs.
- The firewall allows SSH, HTTP (80), and HTTPS (443).
- MySQL port 3306 is not exposed publicly.

2. Prepare the domain

- Choose the production hostname for the application, such as `app.example.com`.
- Create an A record pointing that hostname to the VPS public IPv4 address.
- Wait for DNS propagation.
- Confirm the hostname resolves to the VPS before requesting HTTPS.
- Do not expose MySQL or internal application ports through public DNS.

3. Upload the application

Upload these items to a dedicated directory such as `/opt/nitevow`:

- The complete `Dandy-S00/nitevow` repository.
- `Dockerfile`.
- `docker-compose.yml`.
- Any required patch files, including the `patches` directory.
- A production `.env` file, created directly on the VPS and never committed to Git.

The repository is a pnpm-based TypeScript/Vite and Express application. Its build scripts are:

- `pnpm build` for the production build.
- `pnpm start` for the compiled server.
- `pnpm db:push` for Drizzle/MySQL migrations.

4. Create secure environment variables

Create `/opt/nitevow/.env` with strong, unique values:

```
MYSQL_DATABASE=nitevow
MYSQL_USER=nitevow_app
MYSQL_PASSWORD=generate-a-long-random-password
MYSQL_ROOT_PASSWORD=generate-a-different-long-random-password
```

Also inspect the server code for required application variables, such as:

- Session or cookie secrets.
- Authentication provider credentials.
- Object-storage credentials if file uploads are enabled.
- Email provider credentials if transactional email is enabled.
- OAuth redirect URLs.
- Production application URL.

Never paste secrets into chat, commit them to Git, or place them in a public image.

5. Check the Dockerfile before building

- Confirm the build stage installs from `pnpm-lock.yaml` with a frozen lockfile.
- Confirm the build command completes successfully.
- Confirm the runtime image contains `dist` and the runtime dependencies.
- Confirm the application listens on `0.0.0.0`, not only on `localhost`.
- Confirm the application port matches the Compose configuration: `3000`.
- Confirm the runtime image has a supported way to run Drizzle migrations.

Important: the current Dockerfile prunes development dependencies. Since `drizzle-kit` is a development dependency, `pnpm db:push` will not work inside the final runtime image unless the image is adjusted. Use one of these approaches:

- Add a dedicated migration image or Compose migration service that retains `drizzle-kit`.
- Run migrations from the build stage against the production database, only after confirming the database is reachable and backups exist.
- Build a separate migration target that includes development dependencies, then start the smaller runtime image.

Do not start the production app until the migration strategy is tested.

6. Build and start MySQL

From `/opt/nitevow`:

```
docker compose pull db
docker compose up -d db
docker compose ps
docker compose logs --tail=100 db
```

- Confirm MySQL reports healthy.
- Confirm the named volume `nitevow_mysql_data` exists.
- Confirm the database and application user were created.
- Confirm the database is not published on port 3306.

7. Run database migrations

Use the selected migration strategy from step 5.

- Set `DATABASE_URL` to the internal Compose address, using `db` as the hostname.
- Run `pnpm db:push` once against the new database.
- Review migration output for errors.
- Back up the database before applying future schema changes.
- Never use destructive schema commands against production without a backup and a rollback plan.

The connection format is conceptually:

```
mysql://MYSQL_USER:MYSQL_PASSWORD@db:3306/MYSQL_DATABASE
```

URL-encode special characters in the username or password when constructing `DATABASE_URL`.

8. Build and start the application

```
docker compose build --pull app
docker compose up -d app
docker compose ps
docker compose logs --tail=200 app
```

- The app container remains running.
- The logs show the production server listening successfully.
- The app can connect to MySQL.
- Registration and login work.
- A test account can be created without exposing administrative features.
- File uploads, email, and external integrations work if enabled.

9. Configure Traefik and HTTPS

- Attach the app to the Traefik network already used on the VPS, or configure Traefik to watch the Compose project network.
- Add a router for the production hostname.
- Route traffic to the app container's internal port 3000 .
- Configure the HTTPS entrypoint and ACME certificate resolver.
- Redirect HTTP to HTTPS.
- Add the correct production hostname to authentication callback and cookie settings.
- Do not expose the database or administrative-only services through Traefik.

Before issuing the certificate:

- DNS points to the VPS.
- Port 80 is reachable from the internet.
- Port 443 is reachable from the internet.
- No other service is occupying the hostname.

10. Validate the live deployment

- Open the HTTPS hostname.
- Confirm HTTP redirects to HTTPS.
- Confirm the certificate is valid and matches the hostname.
- Test a new user registration.
- Test login and logout.
- Test an authenticated database write and read.
- Test invalid credentials and authorization boundaries.
- Test password reset or account recovery if implemented.
- Test moderation and administrative routes separately.
- Check browser console and server logs for errors.
- Confirm no secrets appear in responses, logs, or client-side JavaScript.

11. Backups and operations

- Enable scheduled VPS backups if available.
- Back up the MySQL volume or perform regular logical database dumps.
- Store backups outside the VPS.
- Test restoring a backup before relying on it.
- Record the exact Git commit used for production.
- Pin image versions instead of using unreviewed `latest` tags.
- Configure log rotation so Docker logs cannot fill the disk.
- Document how to stop, restart, update, and roll back the application.

Useful operational commands:

```
docker compose ps
docker compose logs --tail=200 app
docker compose logs --tail=200 db
docker compose restart app
docker compose down
```

Do not run `docker compose down -v` in production: removing the volume deletes the MySQL data.

12. Update and rollback procedure

For an update:

1. Record the currently deployed Git commit.
2. Back up MySQL.
3. Review dependency and schema changes.
4. Build the new image.
5. Run migrations using the tested migration process.
6. Restart the app.
7. Run the validation checks.
8. Keep the previous image available until the new release is verified.

For a rollback:

1. Stop the app if the release is unsafe.
2. Restore the previous application image or commit.
3. Restore the database only if the schema change cannot work with the previous application.
4. Verify login, database access, and HTTPS again.